

西安电子科技大学



操作系统实验报告

学院专业 计算机科学与技术

班级 2303013

学号 23009201247

学生姓名 郑墨涵

2025年6月10日

1. 实验目的

[描述实验的主要学习目标]

[列出要掌握的核心概念]

2. 实验原理

[详细描述实验涉及的理论知识]

3. 实验内容

[任务描述+实现步骤]

4. 实验结果

5. 实验总结

6. 实验代码

[代码格式]

```
// 主要函数实现  
int main_function() {  
    // 函数实现代码  
    return 0;  
}
```

实验 1：创建进程

一、实验目的

主要学习目标：学习创建线程实现多工作同步运行；
了解线程与进程之间的数据共享关系。

核心概念：

进程创建 (CreateProcess)

进程同步 (WaitForSingleObject)

进程间通信 (文件系统共享数据)

二、实验原理

1. 进程：

进程是程序的一次执行，拥有独立的内存空间和系统资源

Windows 中每个进程包含至少一个线程（主线程）

2. 进程创建机制：

CreateProcess API：创建新进程及其主线程

参数解析：

lpApplicationName：可执行文件路径

dwCreationFlags：控制标志（如 CREATE_NEW_CONSOLE 创建新控制台）

返回 PROCESS_INFORMATION 包含进程/线程句柄和 ID

3. 进程同步原理：

WaitForSingleObject：阻塞调用进程直到指定对象（如进程句柄）进入信号状态

进程句柄信号规则：进程终止时其句柄变为信号状态

4. 进程间通信 (IPC)：

文件系统作为共享媒介：子进程写入→父进程读取

文件访问同步：通过关闭文件句柄确保写入完成

5. 并行执行模型：

父进程在等待期间执行累加计算，演示 CPU 时间片分配

Windows 调度器管理进程切换

三、实验内容

任务描述：

创建两个进程，让子进程读取一个文件，父进程等待子进程读取完文件后继续执行，实现进程协同工作。

进程协同工作就是协调好两个进程，使之安排好先后次序并以此执行，可以用等待函数来实现这一点。当需要等待子进程运行结束时，可在父进程中调用等待函数

实现步骤：

子进程：

创建/打开 D:\test.txt

写入两行文本数据

读取并打印文件内容

通过 system("pause") 模拟耗时操作

父进程：

创建子进程 (child.exe)

执行 1-100 累加计算（等待期间并行任务）

等待子进程结束

读取并打印子进程生成的文件内容

四、实验结果

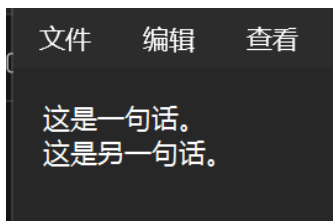
1. 控制台输出：



```
D:\father.exe
Now, sum = 2628
Now, sum = 2701
Now, sum = 2775
Now, sum = 2850
Now, sum = 2926
Now, sum = 3003
Now, sum = 3081
Now, sum = 3160
Now, sum = 3240
Now, sum = 3321
Now, sum = 3403
Now, sum = 3486
Now, sum = 3570
Now, sum = 3655
Now, sum = 3741
Now, sum = 3828
Now, sum = 3916
Now, sum = 4005
Now, sum = 4095
Now, sum = 4186
Now, sum = 4278
Now, sum = 4371
Now, sum = 4465
Now, sum = 4560
Now, sum = 4656
Now, sum = 4753
Now, sum = 4851
Now, sum = 4950
Now, sum = 5050

D:\child.exe
子进程开始运行...
文件打开成功!
写入数据成功!
当前文件内容如下:
这是一句话。
这是另一句话。
请按任意键继续...
```

2. 生成文件内容 (D:\test.txt)：



```
文件  编辑  查看
{
    这是一句话。
    这是另一句话。
}
```

五、实验总结

成功实现父子进程协同工作，验证了进程同步和文件共享机制的有效性。通过等待函数确保执行顺序，文件系统作为数据传递桥梁，符合进程协同设计预期。

六、实验代码

子进程程序 (child.c)

```

#include<stdio.h>
#include<windows.h>
int main()
{
    printf("子进程开始运行...\n\n");
    const char *something = "这是一句话。";
    FILE *fp;
    if(fp = fopen("D:\\test.txt", "w+"))           // 创建/打开文件
    {
        printf("文件打开成功!\n\n");
        fwrite(something, strlen(something), 1, fp); // 写入数据
        fwrite("\n这是另一句话。", strlen("\n这是另一句话。"), 1, fp);
        printf("写入数据成功!\n\n");
        fclose(fp);
        fp = fopen("D:\\test.txt", "r");           // 重新打开读取
        printf("当前文件内容如下: \n");
        char ch=fgetc(fp);
        while(ch!=EOF)                             // 逐字符读取
        {
            putchar(ch);
            ch=fgetc(fp);
        }
        fclose(fp);
    }
    else
        printf("创建文件失败!\n");
    printf("\n\n");
    system("pause");
    return 0;
}

```

父进程程序 (father.c)

```

#include<stdio.h>
#include<windows.h>
int main()
{
    STARTUPINFO sui;           //启动信息结构体
    PROCESS_INFORMATION pi;     //在创建进程时相关的数据结构之一，该结构返回有关新进程及其主线程的信息。
    ZeroMemory(&sui, sizeof(sui));
    sui.cb = sizeof(STARTUPINFO); //将cb成员设置为信息结构体的大小
    int sum = 0;
    char content[100] = "";      //初始化content字符数组用来存放文件内容
    if(CreateProcess("D:\\child.exe", NULL, NULL, NULL, FALSE, CREATE_NEW_CONSOLE, NULL, NULL, &sui, &pi))//创建进程
    {
        printf("已创建一个子进程\n");
        for(int i = 1; i <= 100; i++)
        {
            sum = sum + i;       //求1-100之和
            Sleep(5);            //延迟时间5ms
            printf("Now, sum = %d\n", sum);
        }
        WaitForSingleObject(pi.hProcess, INFINITE); //一直等下去直到进程结束
        FILE *fp = fopen("D:\\test.txt", "r");
        fread(content, sizeof(char), 100, fp);      //设置读取文件内容的相关参数
        printf("子进程创建的文件内容如下:\n\n%s\n\n", content);
        fclose(fp);
    }
    else
        printf("创建子进程失败\n");
    printf("实验结束! ");
    return 0;
}

```

实验 2：线程共享进程数据

一、实验目的

主要学习目标：了解线程与进程之间的数据共享关系。

创建一个线程，在线程中更改进程中的数据。

核心概念：

线程创建 (CreateThread)：参数说明：

第 1 个参数：安全属性 (NULL 默认)

第 2 个参数：栈大小 (0 表示默认)

第 3 个参数：线程函数指针

第 4 个参数：传递给线程函数的参数

第 5 个参数：创建标志 (0 表示立即运行)

第 6 个参数：线程 ID (NULL 表示不需要)

全局变量共享：static int count 是全局变量，被主线程和子线程共享，主线程初始化 count 为 20，子线程修改 count 为 1,3,5,7,9,11。

线程同步 (WaitForSingleObject)：

WaitForSingleObject 使主线程等待子线程结束。INFINITE 表示无限期等待。CloseHandle 释放线程资源。

二、实验原理

1. 线程与进程关系：

线程是 CPU 调度的基本单位，共享进程的内存空间

一个进程包含多个线程，共享代码段/数据段/堆空间

2. 线程创建机制：

CreateThread API：创建新线程

线程函数：DWORD WINAPI ThreadProc (LPVOID lpParameter)

线程终止：正常返回或 ExitThread

3. 内存共享模型：

全局变量存储于进程数据段，所有线程可见

静态变量 (static) 具有进程级生命周期

4. 线程同步机制：

WaitForSingleObject：主线程等待子线程结束

隐式同步：因顺序执行无需额外同步原语

潜在竞态条件：多线程并发访问共享数据需互斥

5. 线程调度：

Windows 时间片轮转调度 (约 20ms)

Sleep() 主动放弃剩余时间片

三、实验内容

任务描述：在进程中定义全局共享数据，在线程中直接引用该数据进行更改并输出该数据。

实现步骤：

定义全局变量 static int count

主线程初始化 count=20

创建子线程执行循环：

主线程阻塞等待子线程结束

主线程打印最终 count 值

四、实验结果

```
进程运行！
进程count=20
新线程运行！
Now,线程count = 1
Now,线程count = 3
Now,线程count = 5
Now,线程count = 7
Now,线程count = 9
线程等待3秒钟...
新线程结束！
进程结束！
Now,count = 11
-----
```

五、实验总结

全局变量在进程内的所有线程间共享，子线程对共享数据的修改会直接影响主线程。通过 WaitForSingleObject 可实现线程间同步，确保执行顺序。最终 count 值验证了线程间数据共享的有效性。

六、实验代码

```
#include<stdio.h>
#include<windows.h>
static int count; // 定义全局共享数据

// 线程函数
DWORD WINAPI ThreadProc(LPVOID IpParameter)
{
    printf("新线程运行！\n\n");

    // 子线程修改共享数据count
    for(count=1; count<=10; count=count+2)
    {
        printf("Now,线程count = %d\n\n",count);
    }

    printf("线程等待3秒钟...\n\n");
    Sleep(3000); // 线程休眠3秒
    return 0;
}

int main()
{
    count=20; // 主线程初始化共享数据
    printf("进程运行！\n\n进程count=%d\n\n",count);

    // 创建新线程
    HANDLE hEvent=CreateThread(NULL,0,ThreadProc,NULL,0,NULL);

    // 主线程等待子线程结束
    WaitForSingleObject(hEvent,INFINITE);
    CloseHandle(hEvent); // 关闭线程句柄

    printf("新线程结束！\n");
    printf("进程结束！\n\n");
    printf("Now,count = %d",count); // 输出最终count值
    return 0;
}
```

实验 3：信号通信

一、实验目的

主要学习目标：利用信号通信机制在父子进程及兄弟进程间进行通信。

核心概念：

命名事件对象 (CreateEvent/OpenEvent)

事件信号 (SetEvent)

等待函数 (WaitForSingleObject)

二、实验原理

1. 事件对象特性：

内核同步对象，有信号/无信号两种状态

类型区分：

手动重置：需显式重置状态

自动重置：触发后自动恢复无信号

2. 命名事件机制：

全局名称空间：“Global\”或“Local\”前缀

跨进程访问：通过相同名称打开事件

3. 事件生命周期：

CreateEvent：创建/打开命名事件

SetEvent：设置事件为有信号状态

ResetEvent：重置为无信号状态

4. 等待机制：

WaitForSingleObject：阻塞等待事件信号

超时控制：INFINITE 或毫秒级超时

返回状态：WAIT_OBJECT_0(成功)/WAIT_TIMEOUT/WAIT_FAILED

5. 进程同步模式：

通知模型：子进程通过事件通知父进程

生产者-消费者变体：事件作为完成信号

三、实验内容

任务描述：父进程创建一个有名事件，由子进程发送事件信号，父进程获取事件信号后进行相应的处理。

实现步骤：

父进程：

创建命名事件“myEvent”（自动重置）

创建子进程 child1.exe

调用 WaitForSingleObject 等待事件

子进程：

打开“myEvent”事件

提示用户输入 y/n

若输入 y 则调用 SetEvent 发送信号

四、实验结果


```
C:\Users\郑墨涵\OneDrive\De × + v
这是一个子进程获得事件信号
-----
Process exited after 4.48 seconds with return value 0
请按任意键继续. . . |

C:/Users/郑墨涵/OneDrive/De × + v
Signal the event to Parent[y/n]y
请按任意键继续. . . |
```

五、实验总结

成功实现基于命名事件的进程间同步机制。通过事件对象的状态变化，子进程可以向父进程发送信号通知事件完成，父进程通过等待函数实现阻塞式同步。实验验证了自动重置事件在跨进程通信中的有效性，以及超时机制在系统健壮性中的重要作用。

六、实验代码

父进程程序（parent1.cpp）

```
#include <stdio.h>
#include <windows.h>
using namespace std;

int main() {
    STARTUPINFO sui;
    ZeroMemory(&sui, sizeof(sui));
    sui.cb = sizeof(STARTUPINFO);
    PROCESS_INFORMATION pi;
    if (!CreateProcess("C:/Users/PC/Desktop/child1.exe", NULL, NULL, NULL, FALSE,
        CREATE_NEW_CONSOLE, NULL, NULL, &sui, &pi))
        printf("进程创建失败");
    printf("这是一个子进程");
    HANDLE hEvent = CreateEvent(NULL, FALSE, TRUE, "myEvent");
    ResetEvent(hEvent);
    int time = 8000;
    DWORD flag = WaitForSingleObject(hEvent, time);
    if (WAIT_FAILED == flag)
        printf("等待时间信号失败");
    else if (WAIT_OBJECT_0 == flag)
        printf("获得事件信号");
    else if (WAIT_TIMEOUT == flag)
        printf("等待子进程信号超时");
    return 0;
}
```

子进程程序（child1.cpp）

```
#include <stdio.h>
#include <windows.h>
using namespace std;

int main() {
    HANDLE hEvent = OpenEvent(EVENT_ALL_ACCESS, TRUE, "myEvent");
    Sleep(1000);
    char ch;
    printf("Signal the event to Parent[y/n]");
    scanf("%c", &ch);
    if (ch == 'y')
        SetEvent(hEvent);
    Sleep(1000);
    system("pause");
    return 0;
}
```

实验 4：匿名管道通信

一、实验目的

主要学习目标：学习使用匿名管道在两个进程间建立通信。

核心概念：

管道创建 (CreatePipe)

句柄继承与重定向

单向数据流 (父→子)

二、实验原理

1. 管道通信模型：

单向通信：数据流从写端到读端，字节流模式：无消息边界。

2. 匿名管道特性：

无名称标识，仅限父子进程间继承，创建时返回两个句柄：读句柄和写句柄。

3. 句柄继承机制：

SECURITY_ATTRIBUTES: bInheritHandle=TRUE 允许继承，子进程继承标准输入/输出重定向。

4. 重定向原理：

STARTUPINFO 结构：hStdInput/hStdOutput 重定向，子进程通过 GetStdHandle(STD_INPUT_HANDLE) 获取管道。

5. 流控制机制：

写端关闭触发读端 EOF，缓冲区满时 WriteFile 阻塞。

三、实验内容

任务描述：分别建立名为 **Parent** 的单文档应用程序和 **Child** 的单文档应用程序作为父子进程，由父进程创建一个匿名管道，实现父子进程向匿名管道写入和读取数据。

实现步骤：

父进程：

创建管道 (CreatePipe)

设置子进程标准输入为管道读端

向管道写入数据 "Hello from Parent!"

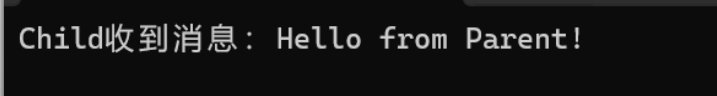
关闭写句柄触发子进程读取结束

子进程：

从标准输入读取数据

打印接收到的消息

四、实验结果



Child收到消息: Hello from Parent!

五、实验总结

匿名管道是实现父子进程间单向通信的有效机制。通过合理管理管道句柄和进程标准输入输出重定向，可以构建高效的进程间数据通道。本实验成功演示了父进程向子进程发送数据的基本流程，验证了 Windows 管道 API 的实际应用价值。

六、实验代码

父进程程序 (Parent.cpp)

```

#include <windows.h>
#include <stdio.h>
#include <string.h>

int main() {
    SECURITY_ATTRIBUTES sa = {sizeof(SEcurity_ATTRIBUTES), NULL, TRUE};

    HANDLE hRead = NULL, hWrite = NULL;
    if (!CreatePipe(&hRead, &hWrite, &sa, 0)) {
        printf("CreatePipe 失败, 错误码: %u\n", GetLastError());
        return 1;
    }

    SetHandleInformation(hWrite, HANDLE_FLAG_INHERIT, 0);

    STARTUPINFOA si = {0};
    si.cb = sizeof(si);
    si.dwFlags = STARTF_USESTDHANDLES;
    si.hStdInput = hRead;
    si.hStdOutput = GetStdHandle(STD_OUTPUT_HANDLE);
    si.hStdError = GetStdHandle(STD_ERROR_HANDLE);

    PROCESS_INFORMATION pi = {0};
    char cmdLine[] = "Child.exe";

    if (!CreateProcessA(
        NULL,
        cmdLine,
        NULL, NULL,
        TRUE,
        0,
        NULL, NULL,
        &si, &pi)) {
        printf("CreateProcess 失败, 错误码: %u\n", GetLastError());
        return 1;
    }

    CloseHandle(hRead);

    const char *message = "Hello from Parent!";
    DWORD written = 0;
    WriteFile(hWrite, message, (DWORD)strlen(message), &written, NULL);

    CloseHandle(hWrite);

    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);

    return 0;
}

```

子进程程序 (child.cpp)

```

#include <windows.h>
#include <stdio.h>

int main() {
    char buffer[128] = {0};
    DWORD bytesRead = 0;

    if (ReadFile(
        GetStdHandle(STD_INPUT_HANDLE),
        buffer, sizeof(buffer) - 1,
        &bytesRead, NULL)
        && bytesRead > 0) {
        buffer[bytesRead] = '\0';
        printf("Child收到消息: %s\n", buffer);
    }

    return 0;
}

```

实验 5：命名管道通信

一、实验目的

主要学习目标：学习使用命名匿名管道在多进程间建立通信。

核心概念：

命名管道服务器 (CreateNamedPipe)

客户端连接 (CreateFile)

双向通信支持

二、实验原理

1. 命名管道架构：

客户端/服务器模型

唯一路径标识：\\.\pipe\PipeName

支持跨网络通信

2. 服务器端操作：

CreateNamedPipe：创建管道实例

参数控制：

PIPE_ACCESS_INBOUND：单向接收

PIPE_TYPE_BYTE：字节流模式

ConnectNamedPipe：监听客户端连接

3. 客户端操作：

CreateFile：连接已存在管道

OPEN_EXISTING：打开模式

GENERIC_WRITE：写权限请求

4. 通信协议：

面向连接：需显式建立连接

消息边界支持：PIPE_TYPE_MESSAGE

阻塞/非阻塞模式：PIPE_WAIT/PIPE_NOWAIT

5. 错误处理机制：

常见错误：

ERROR_PIPE_BUSY：实例数超限

ERROR_FILE_NOT_FOUND：管道未创建

WaitNamedPipe：客户端等待管道可用

三、实验内容

任务描述：建立父子进程，由父进程创建一个命名匿名管道，由子进程向命名管道写入数据，由父进程从命名管道读取数据。

实现步骤：

父进程：

创建命名管道 (PIPE_ACCESS_INBOUND)

启动子进程 Child2.exe

阻塞等待连接 (ConnectNamedPipe)

读取并打印管道数据

子进程：

连接命名管道 (OPEN_EXISTING)

发送消息 "Hello from Child via named pipe"

四、实验结果

```
Parent从命名管道收到: Hello from Child via named pipe
```

五、实验总结

命名管道是实现跨进程通信的有效机制，特别适用于客户端/服务器模式的交互场景。本实验成功演示了父进程（服务器）创建管道、子进程（客户端）连接并发送数据、父进程接收数据的完整流程，验证了 Windows 命名管道 API 的实际应用价值。相比匿名管道，命名管道具有更灵活的应用范围和更强的通信能力。

六、实验代码

父进程程序（Parent.cpp）

```
#include <windows.h>
#include <stdio.h>
#include <string.h>

int main()
{
    const char *PIPE_NAME = "\\.\pipe\\MyPipe";

    HANDLE hPipe = CreateNamedPipe(
        PIPE_NAME,
        PIPE_ACCESS_INBOUND,
        PIPE_TYPE_BYTE | PIPE_WAIT,
        1,
        0, 0,
        0,
        NULL);

    if (hPipe == INVALID_HANDLE_VALUE) {
        printf("CreateNamedPipe 失败, Error= %u\n", GetLastError());
        return 1;
    }

    STARTUPINFOA si = {sizeof(si)};
    PROCESS_INFORMATION pi = {0};

    if (!CreateProcess(
        NULL,
        (LPSTR)"Child2.exe",
        NULL, NULL,
        FALSE,
        0, NULL, NULL,
        &si, &pi)) {
        printf("CreateProcess 失败, Error=%u\n", GetLastError());
        CloseHandle(hPipe);
        return 1;
    }

    BOOL connected = ConnectNamedPipe(hPipe, NULL);
    if (!connected) {
        printf("ConnectNamedPipe 失败, Error=%u\n", GetLastError());
        CloseHandle(hPipe);
        return 1;
    }
}
```

子进程程序（Child2.cpp）

```
#include <windows.h>
#include <stdio.h>
#include <string.h>

int main() {

    const char *PIPE_NAME = "\\.\pipe\MyPipe";

    HANDLE hPipe = CreateFileA(
        PIPE_NAME,
        GENERIC_WRITE,
        0,
        NULL,
        OPEN_EXISTING,
        0, NULL);

    if (hPipe == INVALID_HANDLE_VALUE) {
        printf("CreateFile 打开管道失败, Error= %u\n", GetLastError());
        return 1;
    }
    const char *message = "Hello from Child via named pipe";
    DWORD written = 0;
    if (!WriteFile(hPipe, message, (DWORD)strlen(message), &written, NULL)) {
        printf("WriteFile 失败, Error=%u\n", GetLastError());
    }

    CloseHandle(hPipe);
    return 0;
}
```

实验 6：信号量实现进程同步

一、实验目的

主要学习目标：

核心概念：

共享内存 (CreateFileMapping)

信号量 (CreateSemaphore)

互斥锁 (CreateMutex)

二、实验原理

1. 经典同步问题：

生产者：生成数据放入缓冲区

消费者：从缓冲区取出数据

缓冲区：有限共享资源

2. 同步原语：

信号量 (Semaphore)：

计数信号量：SEM_EMPTY (空槽位) / SEM_FULL (满槽位)

P 操作 (WaitForSingleObject)：申请资源

V 操作 (ReleaseSemaphore)：释放资源

互斥锁 (Mutex)：

二进制锁：保护临界区

WaitForSingleObject 获取锁

ReleaseMutex 释放锁

3. 共享内存机制：

CreateFileMapping：创建命名共享内存

MapViewOfFile：映射内存视图

读写一致性：无锁机制下需同步原语保证

4. 环形缓冲区实现：

```
struct {  
    int buffer[BUFFER_SIZE];  
    int in; // 生产者指针  
    int out; // 消费者指针  
}
```

指针计算：(index + 1) % BUFFER_SIZE

满/空条件：

空：in == out

满：(in + 1) % BUFFER_SIZE == out

5. 进程间同步协议：

确保：缓冲区满时生产者阻塞

缓冲区空时消费者阻塞

互斥访问缓冲区

三、实验内容

任务描述：

生产者进程生产产品，消费者进程消费产品。

当生产者进程生产产品时，如果没有空缓冲区可用，那么生产者进程必须等待消费者进程释放出一个缓冲区。

当消费者进程消费产品时，如果缓冲区中没有产品，那么消费者进程将被阻塞，直到新的产品被生产出来

实现步骤：

生产者：

创建共享内存和同步对象

启动消费者进程

循环生产：

等待空位 (semEmpty--)

获取互斥锁 → 写入缓冲区 → 释放锁

增加满位 (semFull++)

消费者：

打开共享内存和同步对象

循环消费：

等待满位 (semFull--)

获取互斥锁 → 读取缓冲区 → 释放锁

增加空位 (semEmpty++)

四、实验结果

```
producer: waiting for empty slot...
producer: got empty slot
producer put 1 at [0]
Producer: signaled full slot
producer: waiting for empty slot...
producer: got empty slot
producer put 2 at [1]
Producer: signaled full slot
consumer: waiting for full slot...
consumer: got full slot
consumer got 1 from [0]
consumer: signaled empty slot
producer: waiting for empty slot...
producer: got empty slot
producer put 3 at [2]
Producer: signaled full slot
consumer: waiting for full slot...
consumer: got full slot
consumer got 2 from [1]
consumer: signaled empty slot
producer: waiting for empty slot...
producer: got empty slot
producer put 4 at [3]
```

五、实验总结

通过共享内存和同步对象（信号量+互斥锁）成功实现了生产者-消费者模型。该方案有效解决了进程间数据共享和同步问题，确保生产者和消费者在缓冲区操作中的正确协作。

六、实验代码

生产者程序（producer.cpp）

```
#include <windows.h>
#include <stdio.h>

#define BUFFER_SIZE 5
#define PRODUCE_COUNT 10

const char *SHM_NAME = "ProdconsShm";
const char *SEM_EMPTY = "SemEmpty";
const char *SEM_FULL = "SemFull";
const char *MUTEX_NAME = "MutexBuffer";

struct SharedBuffer {
    int buffer[BUFFER_SIZE];
    int in, out;
};

int main() {
    HANDLE hMap = CreateFileMappingA(
        INVALID_HANDLE_VALUE, NULL,
        PAGE_READWRITE,
        0, sizeof(SharedBuffer),
        SHM_NAME
    );

    if (!hMap) {
        printf("CreateFileMapping Failed %u\n", GetLastError());
        return 1;
    }
    SharedBuffer *shm = (SharedBuffer *)MapViewOfFile(
        hMap, FILE_MAP_ALL_ACCESS, 0, 0, sizeof(SharedBuffer)
    );
    shm->in = shm->out = 0;

    HANDLE semEmpty = CreateSemaphoreA(NULL, BUFFER_SIZE, BUFFER_SIZE, SEM_EMPTY);
    HANDLE semFull = CreateSemaphoreA(NULL, 0, BUFFER_SIZE, SEM_FULL);
    HANDLE hMutex = CreateMutexA(NULL, FALSE, MUTEX_NAME);
    if (!semEmpty || !semFull || !hMutex) {
        printf("CreateSemaphore/Mutex Failed %u\n", GetLastError());
        return 1;
    }
}
```

```

STARTUPINFOA si = {sizeof(si)};
PROCESS_INFORMATION pi = {0};
if (!CreateProcessA(
    NULL,
    (LPSTR) "Consumer.exe",
    NULL, NULL,
    FALSE,
    0, NULL, NULL,
    &si, &pi)) {
    printf ("CreateProcess Failed %u\n", GetLastError());
    return 1;
}

for (int item = 1; item <= PRODUCE_COUNT; ++item) {
    printf("producer: waiting for empty slot...\n");
    WaitForSingleObject (semEmpty, INFINITE);
    printf("producer: got empty slot\n");

    WaitForSingleObject (hMutex, INFINITE);
    shm->buffer[shm->in] = item;
    printf("producer put %d at [%d]\n", item, shm->in);
    shm->in = (shm->in + 1) % BUFFER_SIZE;
    ReleaseMutex (hMutex);

    ReleaseSemaphore (semFull, 1, NULL);
    printf("Producer: signaled full slot\n");

    Sleep(100);
}

WaitForSingleObject (pi.hProcess, INFINITE);

CloseHandle (pi.hProcess);
CloseHandle (pi.hThread);
CloseHandle (semEmpty);
CloseHandle (semFull);
CloseHandle (hMutex);

UnmapViewOfFile (shm);
CloseHandle (hMap);
return 0;
}

```

消费者程序（consumer.cpp）

```

#include <windows.h>
#include <stdio.h>

#define BUFFER_SIZE    5
#define PRODUCE_COUNT 10

const char *SHM_NAME   = "ProdconsShm";
const char *SEM_EMPTY  = "SemEmpty";
const char *SEM_FULL   = "SemFull";
const char *MUTEX_NAME = "MutexBuffer";

struct SharedBuffer {
    int buffer[BUFFER_SIZE];
    int in, out;
};

int main() {
    HANDLE hMap = OpenFileMappingA(
        FILE_MAP_ALL_ACCESS,
        FALSE,
        SHM_NAME
    );
}

```

```

if (!hMap) {
    printf("OpenFileMapping Failed %u\n", GetLastError());
    return 1;
}
SharedBuffer *shm = (SharedBuffer *)MapViewOfFile(
    hMap, FILE_MAP_ALL_ACCESS, 0, 0, sizeof(SharedBuffer)
);

HANDLE semEmpty = OpenSemaphoreA (SEMAPHORE_ALL_ACCESS, FALSE, SEM_EMPTY);
HANDLE semFull = OpenSemaphoreA(SEMAPHORE_ALL_ACCESS, FALSE, SEM_FULL);
HANDLE hMutex = OpenMutexA (MUTEX_ALL_ACCESS, FALSE, MUTEX_NAME);
if (!semEmpty || !semFull || !hMutex) {
    printf("OpenSemaphore/Mutex Failed %u\n", GetLastError());
    return 1;
}

for (int i = 1; i <= PRODUCE_COUNT; ++i) {
    printf("consumer: waiting for full slot...\n");
    WaitForSingleObject (semFull, INFINITE);
    printf("consumer: got full slot\n");

    WaitForSingleObject (hMutex, INFINITE);
    int item = shm->buffer[shm->out];
    printf("consumer got %d from [%d]\n", item, shm->out);
    shm->out = (shm->out + 1) % BUFFER_SIZE;
    ReleaseMutex (hMutex);

    ReleaseSemaphore (semEmpty, 1, NULL);
    printf("consumer: signaled empty slot\n");

    Sleep(150);
}

CloseHandle (semEmpty);
CloseHandle (semFull);
CloseHandle (hMutex);

UnmapViewOfFile (shm);
CloseHandle (hMap);
return 0;
}

```